

Introduction to Web and HTML Fundamentals

Building the Foundation of the Web

Chapter Overview

- 01** Introduction to Web Fundamentals
- 02** Getting Started with HTML
- 03** HTML Forms
- 04** More HTML Elements
- 05** HTML Video and Audio



Introduction to Web Fundamentals

Section 1



The Full Stack Overview

What is the Web?

- A system of interconnected documents and resources accessed via browsers
- Built on open standards: HTML, CSS, JavaScript, and HTTP

The Full Stack

- Front-end: what the user sees and interacts with (browser)
- Back-end: server-side logic, APIs, and data processing
- Database: persistent storage for application data
- Infrastructure: hosting, networking, and deployment



Front-End vs Back-End

Front-End (Client Side)

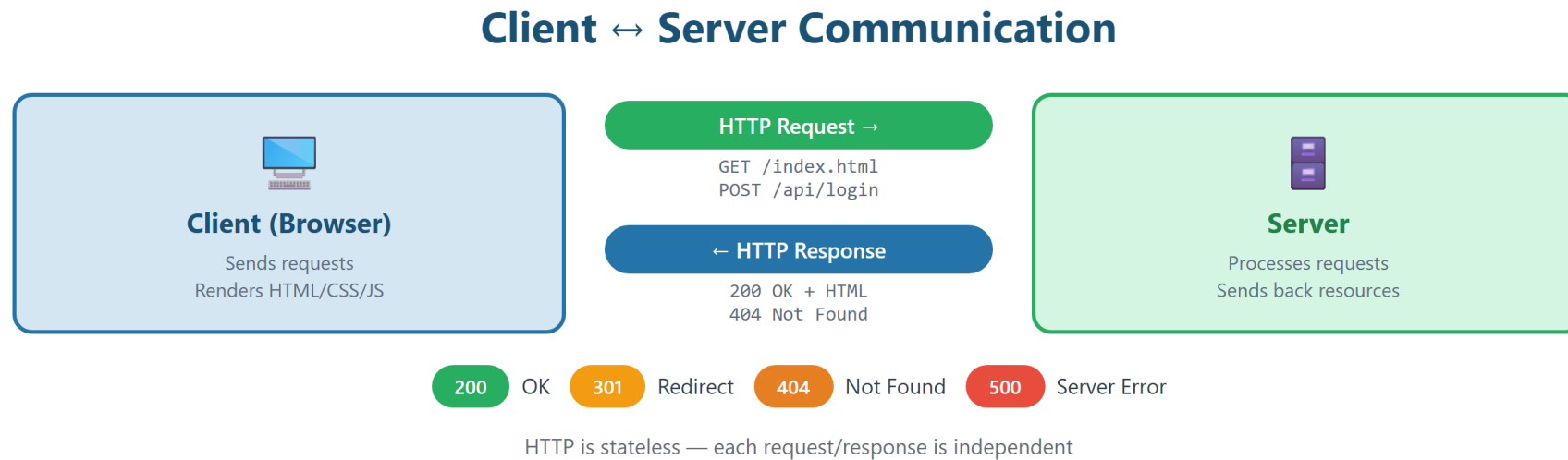
- HTML — structure and content of the page
- CSS — styling, layout, colors, and responsive design
- JavaScript — interactivity, dynamic behavior, and DOM manipulation
- Frameworks: React, Angular, Vue.js

Back-End (Server Side)

- Server languages: Node.js, Python, Java, C#
- APIs: expose data and functionality to the front-end
- Authentication, business logic, and data validation



How the Web Works



Databases and Core Web Skills

Databases

- Store and retrieve structured data (users, products, orders)
- Relational (SQL): MySQL, PostgreSQL — tables with relationships
- NoSQL: MongoDB, DynamoDB — flexible document-based storage

Core Skills for This Course

- HTML — the language every web page is built with
- Understanding how browsers render content
- How HTTP requests and responses work



Modern Web Frameworks

React

- Component-based UI library created by Facebook (Meta)
- Uses JSX — HTML-like syntax inside JavaScript
- One of the most widely used front-end libraries

Why Frameworks Matter

- Build complex UIs from reusable components
- Manage application state efficiently
- All frameworks still produce HTML, CSS, and JS in the browser
- Learning HTML first is essential — it's the foundation



Getting Started with HTML

Section 2



What is HTML?

Definition

- HyperText Markup Language — the standard language for creating web pages
- Describes the structure and content of a document using elements and tags
- Not a programming language — it's a markup language

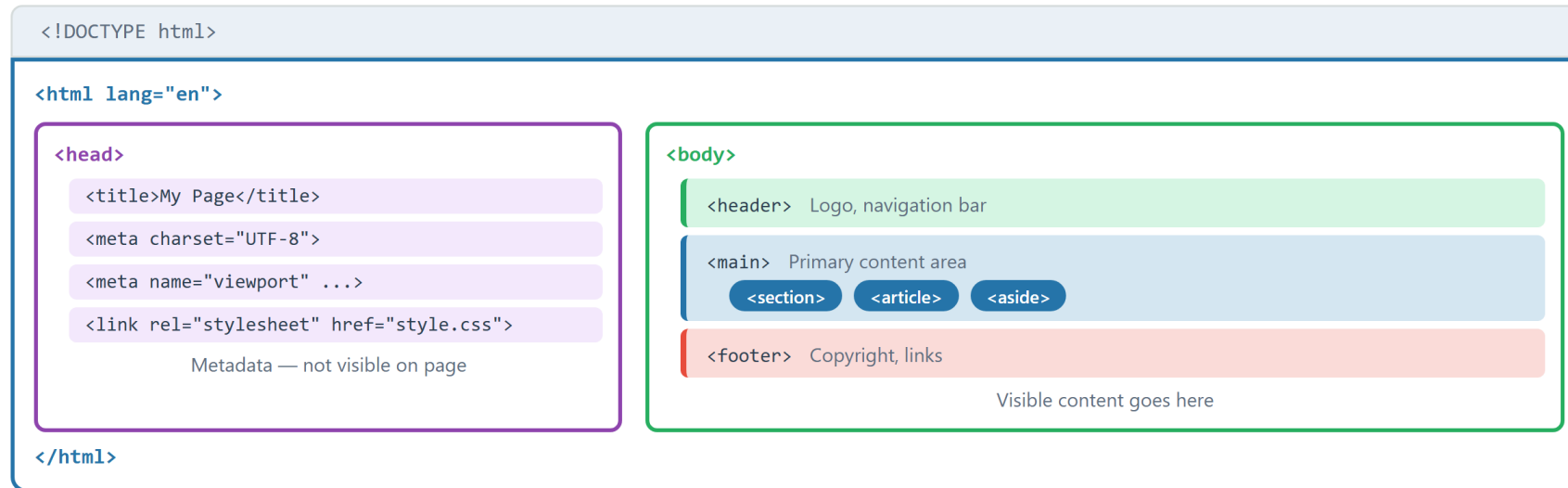
Brief History

- 1991: Tim Berners-Lee created HTML at CERN
- 1999: HTML 4.01 became a W3C recommendation
- 2014: HTML5 — the current standard with multimedia, semantics, and APIs
- HTML is now maintained as a Living Standard by WHATWG



HTML5 Document Structure

HTML5 Document Structure



Your First HTML Page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first web page.</p>
</body>
</html>
```



Anatomy of an HTML Element

Anatomy of an HTML Element

`<a href="https://example.com" class="link"> Click Here </a>`

Opening Tag Attributes (name="value" pairs) Content (text/nested) Closing Tag

Paired Elements

```
<p>Text</p>
<h1>Title</h1>
<div>...</div>
```

Self-Closing (Void)

```

<br>
<input type="text">
```

Nested Elements

```
<ul>
  <li>Item</li>
</ul>
```

Nested HTML Elements

What is Nesting?

- HTML elements can contain other elements inside them
- All HTML documents are built from nested elements forming a tree structure

Document Structure Example

- `<html>` contains `<head>` and `<body>`
- `<body>` contains `<h1>`, `<p>`, `<div>`, and other visible elements
- `<div>` can contain `<p>`, `<ul>`, `<img>`, and more elements inside it
- Proper nesting: always close inner tags before outer tags



Headings and Paragraphs

Headings

- `<h1>` through `<h6>` — six levels of heading importance
- `<h1>` is the main heading (one per page), `<h2>`–`<h6>` for sub-sections
- Headings create a document outline for accessibility and SEO

Paragraphs and Text

- `<p>` — defines a paragraph of text
- `<br>` — line break (self-closing, no end tag)
- `<strong>` — bold/important text, `<em>` — italic/emphasized text

Attributes and Styles

HTML Attributes

- Provide extra information about an element
- Written inside the opening tag: `<tag attribute="value">`
- Common: id, class, style, title, lang

Inline Styles

- Apply CSS directly with the style attribute
- `<p style="color: blue; font-size: 16px;">Styled text</p>`
- Prefer external CSS stylesheets for maintainability



Images and Links

Images

- `` — self-closing element
- alt attribute is required for accessibility (screen readers)
- width and height attributes control display size

Links (Anchors)

- `<a href="https://example.com">Click here</a>`
- href can be absolute URL, relative path, or #id for page anchors
- target="_blank" opens the link in a new tab



Images and Links — Examples

```
<!-- Image with alt text -->


<!-- External link -->
<a href="https://www.w3schools.com">Visit W3Schools</a>

<!-- Link that opens in new tab -->
<a href="https://developer.mozilla.org" target="_blank">
  MDN Web Docs
</a>

<!-- Image as a link -->
<a href="/home">
  
</a>
```

Web Servers and HTTP

Web Servers

- A computer that stores and serves web pages to clients (browsers)
- Popular servers: Apache, Nginx, IIS, Node.js (JavaScript runtime commonly used to build web servers such as Express apps)

HTTP Protocol

- HyperText Transfer Protocol — the rules for web communication
- Request: browser asks for a resource (GET /index.html)
- Response: server sends back HTML, CSS, images, etc.
- Status codes: 200 OK, 404 Not Found, 500 Server Error
- HTTPS adds encryption for secure communication



HTTP Methods (Verbs)

What Are HTTP Methods?

- Every HTTP request includes a method that tells the server what action to perform
- HTML forms use GET and POST — JavaScript (fetch/AJAX) can use all methods

GET

- Retrieve data — the default for links and typing URLs in the browser
- Data sent in the URL query string: /search?q=html

POST

- Send data to the server — used by forms for login, registration, uploads
- Data sent in the request body (not visible in the URL)



More HTTP Methods and Status Codes

Additional Methods (Used in APIs)

- PUT — update/replace an existing resource on the server
- PATCH — partially update a resource (change specific fields)
- DELETE — remove a resource from the server

Common Status Codes

- 200 OK — request succeeded
- 301 Moved Permanently — resource has a new URL (redirect)
- 404 Not Found — the requested resource does not exist
- 500 Internal Server Error — something went wrong on the server



HTML Forms

Section 3



The Form Element

What Are Forms?

- Forms collect user input and send it to a server for processing
- Login pages, search bars, registration, surveys — all use forms

The `<form>` Tag

- Container for all form controls (inputs, buttons, selects)
- action attribute — URL where form data is sent
- method attribute — GET (in URL) or POST (in request body)



HTML Form Elements Overview

HTML Form Elements

Input Types

`<input type="text">` Plain text
`<input type="password">` Hidden input
`<input type="email">` Email validation
`<input type="number">` Numeric only

Selection Controls

`<input type="radio">` One of many
`<input type="checkbox">` Toggle on/off
`<select>...</select>` Dropdown list
`<textarea>...</textarea>` Multi-line

Action Buttons

`<button type="submit">` Send form
`<button type="reset">` Clear form
`<button type="button">` Custom JS

Grouping & Labels

`<form>...</form>` Container
`<fieldset>...</fieldset>` Group fields
`<legend>Title</legend>` Group label
`<label for="id">...</label>` Field label

Always use `<label>` with form controls for accessibility

Text Input Types

Text Input

- `<input type="text">` — single-line text field
- name attribute identifies the field when data is submitted

Password Input

- `<input type="password">` — masks characters as dots

Submit Button

- `<input type="submit" value="Sign In">` — sends the form data
- `<button type="submit">Sign In</button>` — same but more flexible



Selection Input Types

Radio Buttons

- `<input type="radio">` — select one option from a group
- All radios in a group share the same name attribute

Checkboxes

- `<input type="checkbox">` — select multiple options
- Each checkbox can be checked or unchecked independently

Buttons

- `<input type="button" value="Click Me">` — no default behavior
- Use JavaScript onclick to add functionality



Labels, Textarea, and Select

The <label> Element

- Associates text with a form control for accessibility
- Explicit: `<label for="email">` with matching `<input id="email">`
- Implicit: wrap the input inside the label — `<label>Email <input></label>`

Textarea

- `<textarea rows="5" cols="40">Default text</textarea>`
- Multi-line text input — rows and cols set visible size

Dropdown Select

- `<select>` with `<option>` elements creates a dropdown list
- `selected` attribute sets the default choice



Fieldset and Legend

Grouping Form Controls

- `<fieldset>` groups related form elements with a visible border
- `<legend>` provides a caption for the fieldset group
- Improves form organization and accessibility

Best Practices

- Group related inputs (e.g., personal info, address, payment)
- Always pair `<label>` with every input for screen reader support
- Use fieldset for radio button groups and checkbox groups



Registration Form Example

```
<form action="/register" method="POST">
  <fieldset>
    <legend>Create Account</legend>

    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required>

    <button type="submit">Register</button>
  </fieldset>
</form>
```

More HTML Elements

Section 4



Advanced Input Types

Date and Time Inputs

- `<input type="date">` — date picker (calendar widget)
- `<input type="time">` — time selector
- `<input type="month">` — month and year picker

Specialized Inputs

- `<input type="email">` — validates email format automatically
- `<input type="number">` — numeric input with up/down arrows
- `<input type="tel">` — telephone number (mobile keyboards show number pad)
- `<input type="range">` — slider control for numeric ranges



Input Restrictions and Attributes

Numeric Restrictions

- min and max — set allowed range: min="1" max="100"
- step — increment value: step="5" for multiples of 5

Size and Length

- size — visible width of the input field (characters)
- maxlength — maximum number of characters allowed
- minlength — minimum characters required
- These provide client-side validation without JavaScript



Validation Attributes

required

- Form won't submit until the field has a value
- Browser shows a native error message automatically

pattern

- Regex pattern the input must match: `pattern="[A-Za-z]{3,}"`
- Use title attribute to explain the expected format to users

placeholder

- Hint text shown when input is empty: `placeholder="Enter your name"`
- Disappears when the user starts typing



Additional Input Features

autofocus

- Automatically focuses the input when the page loads
- Use on the first important field (e.g., search box, username)

list and <datalist>

- Provides autocomplete suggestions for an input field
- `<input list="colors"> + <datalist id="colors"><option>Red</option>...</datalist>`

reset

- `<input type="reset">` — clears all form fields to their default values
- Use sparingly — users rarely want to erase everything



Validated Form Example

```
<form action="/submit" method="POST">
  <label for="name">Full Name:</label>
  <input type="text" id="name" name="name"
    required minlength="2" placeholder="Jane Doe" autofocus>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required
    placeholder="jane@example.com">

  <label for="age">Age:</label>
  <input type="number" id="age" name="age"
    min="18" max="120" step="1">

  <label for="phone">Phone:</label>
  <input type="tel" id="phone" name="phone"
    pattern="[0-9]{3}-[0-9]{3}-[0-9]{4}"
    title="Format: 123-456-7890">

  <button type="submit">Submit</button>
</form>
```



HTML Video and Audio

Section 5



Multimedia on the Web

Before HTML5

- Required browser plugins like Flash or Silverlight
- Plugins caused security issues and poor mobile support

HTML5 Multimedia

- Native <video> and <audio> elements — no plugins needed
- Supported by all modern browsers

Common Formats

- Video: MP4 (H.264), WebM, Ogg — MP4 has widest support
- Audio: MP3, WAV, Ogg — MP3 is the universal format



Video and Audio Elements

HTML5 Media Elements

 **<video>**

```
<video controls width="400">
  <source src="movie.mp4"
    type="video/mp4">
</video>
```

Formats:

[MP4](#) [WebM](#) [Ogg](#)

 **<audio>** **<iframe>**

Use the controls attribute to show play/pause/volume UI — always provide fallback text

The Video and Audio Elements

Video Attributes

- controls — shows play/pause, volume, and progress bar
- autoplay — starts playing when the page loads
- loop — replays automatically when it ends
- muted — starts without sound (required for autoplay in most browsers)

Audio Attributes

- Same attributes: controls, autoplay, loop, muted
- No visual display — just the player controls
- Use <source> elements for multiple formats as fallback



Video and Audio Examples

```
<!-- Video with controls -->
<video width="640" height="360" controls>
  <source src="movie.mp4" type="video/mp4">
  <source src="movie.webm" type="video/webm">
  Your browser does not support the video tag.
</video>

<!-- Audio player -->
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  <source src="song.ogg" type="audio/ogg">
  Your browser does not support the audio tag.
</audio>

<!-- Autoplay muted video (background video) -->
<video autoplay muted loop>
  <source src="background.mp4" type="video/mp4">
</video>
```


Embedding YouTube Videos

The <iframe> Element

- YouTube provides embed codes using <iframe> — not <video>
- Go to YouTube → Share → Embed → copy the HTML code
- Set width and height for responsive sizing

YouTube Parameters

- autoplay=1 — auto-start the video
- loop=1 — replay continuously
- controls=0 — hide player controls
- start=30 — start playback at 30 seconds



YouTube Embed Example

```
<!-- Basic YouTube embed -->
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/VIDEO_ID"
  title="YouTube video player"
  allow="accelerometer; autoplay; clipboard-write;
    encrypted-media; gyroscope; picture-in-picture"
  allowfullscreen>
</iframe>

<!-- With parameters -->
<iframe width="560" height="315"
  src="https://www.youtube.com/embed/VIDEO_ID?autoplay=1&start=30"
  title="Tutorial Video"
  allowfullscreen>
</iframe>
```



Key Takeaways

Key Takeaways

1. HTML is the foundation of every web page — structure and content
2. HTML5 document structure: `<!DOCTYPE html>`, `<html>`, `<head>`, `<body>`
3. Forms collect user input with `<input>`, `<select>`, `<textarea>` elements
4. Validation attributes (required, pattern, min/max) reduce need for JavaScript
5. Native `<video>` and `<audio>` elements replaced plugins like Flash
6. YouTube embedding uses `<iframe>` — not the `<video>` element

